Match the Git command to the GUI action

A live matching activity for the Git for Science workshop

How this works

In this workshop we drive Git through a graphical interface: buttons in the RStudio Git pane and on GitHub. Every one of those clicks runs a Git command underneath. Clicking the push button *is* `git push`.

Seeing that equivalence matters. Using the interface is not "cheating", it is running the same commands a terminal user types. It is also the bridge to the agentic-coding follow-up, where an agent drives Git by naming these same commands.

The activity (10 minutes, in pairs): the left column lists Git commands, numbered. The right column describes GUI actions, lettered and out of order. Draw a line from each command to its matching action, or write the letter next to the number. Then we compare answers as a group.

Learner sheet

Git command	GUI action (out of order)
1. `git clone <url>`	A. Click the **down-arrow** (pull) button in the RStudio Git pane
2. `git add <file>` / stage	B. Write a commit message, then click **Commit**
3. `git commit -m "..."`	C. On GitHub, click **Add file** > **Create new file** or **Upload files**, then **Commit changes**
4. `git push`	D. Copy the HTTPS URL, then **File** > **New Project** > **Version Control** > **Git** in RStudio
5. `git pull`	E. Click the **up-arrow** (push) button in the RStudio Git pane
6. `git status`	F. Tick the **checkbox** next to a file in the RStudio Git pane
7. `git log`	G. On GitHub, open the **Pull requests** tab and click **New pull request**
8. `git branch <name>` / checkout	H. Read the file list and status flags shown in the RStudio Git pane
9. `git add` + `git commit` (on GitHub)	I. Click the **branch dropdown** in the Git pane and choose **New Branch**
10. open a pull request	J. Click **History** (the clock icon) in the RStudio Git pane

Pairs: 1 ______ 2 ______ 3 ______ 4 ______ 5 ______ 6 ______ 7 ______ 8 ______ 9 ______ 10 ______

Facilitator answer key

Do not print this page for learners.

Command	Answer	GUI action
1. `git clone <url>`	**D**	File > New Project > Version Control > Git
2. `git add` / stage	**F**	Tick the checkbox next to a file in the Git pane
3. `git commit -m "..."`	**B**	Write a message, click Commit
4. `git push`	**E**	Up-arrow (push) button in the Git pane
5. `git pull`	**A**	Down-arrow (pull) button in the Git pane
6. `git status`	**H**	Read the file list and status flags in the Git pane
7. `git log`	**J**	History (clock icon) in the Git pane
8. `git branch` / checkout	**I**	Branch dropdown > New Branch
9. `git add` + `commit` on GitHub	**C**	Add file / Upload files, then Commit changes
10. open a pull request	**G**	Pull requests tab > New pull request

Talking points while reviewing

- Every button press ran a real command. Nothing was hidden or "easier"; it was the same operation.
- `git add` and `git commit` are two steps. In the RStudio pane you see this clearly: tick the box (add), then Commit. On GitHub the web editor bundles both into one "Commit changes" click.
- A pull request is not a Git command in the same sense; it is a GitHub feature built on top of branches you pushed. Good moment to name where Git ends and GitHub begins.
- Tie-in to the agentic follow-up: an agent driving Git speaks these same command names. Knowing the mapping is what lets you read and check what the agent does.